Advanced Algorithms: Exam 2 Review

Dana Randall Spring 2024

Sarthak Mohanty

Exercise 1

Short-answer questions. No justification required.

(a) Consider Fortune's algorithm to compute the Voronoi diagram of n sites in the plane. What is the maximum number of arcs that any one site can contribute to the beach line?

(b) Which of the following assertions about the Delaunay triangulation (DT) of a set P of n points in the plane are always true? Select all that apply.

i. The closest pair of points are connected by an edge in the DT.

ii. The farthest pair of points cannot be connected by an edge in the DT.

iii. Among all triangulations of P, the DT maximizes the minimum angle.

iv. Among all triangulations of P, the DT minimizes the maximum angle.

v. Among all triangulations of P, the DT minimizes the total sum of edge lengths

(c) Recall the Max3SAT problem. The input is a Boolean formula in conjunctive normal form and you wish to output an assignment of the variables that satisfy the maximum number of clauses. True or false: there exists an ϵ-approximation for the Max3SAT problem for any $\epsilon > 0$.

(d) True or false: there exists a 2-approximation for the TSP problem.

Exercise 2

(Erickson) Suppose you are a TA for a computational geometry class. You're allowing the students to use their own favorite programming language and programming environment, which means you can't compile the code yourself. Thus, instead of submitting the code itself, you asked your students to submit several Voronoi diagrams computed by their code

After the submission deadline, you realize that you forgot to ask the students to submit the sites that defined each of their Voronoi diagrams! In frustration, you decide to give your students full credit if every diagram they submit is the Voronoi diagram of some point set. But how do you tell?[1]

[1] I had a similar issue as a TA for 2051.

Describe and analyze an algorithm to determine, given a planar straight-line graph G, whether G is a Voronoi diagram. Your algorithm should either return a finite point set P such that G is the Voronoi diagram of P, or report correctly that no such point set exists. [*Hint:* Consider the case of four sites.]

Exercise 3

(Mount) In class we proved that the Delaunay triangulation of a set of sites in the plane maximizes the minimum angle. (It is the max-min angle triangulation.) Unfortunately, it is not the best triangulation for the following two criteria.

(a) Give an example of a set of point sites in the plane such that the Delaunay triangulation of this set does not minimize the sum of edge lengths, among all possible triangulations. In other words, the Delaunay triangulation is not the minimum-weight triangulation.

(b) Give an example of a set of point sites in the plane such that the Delaunay triangulation of this point set does not minimize the maximum angle, among all possible triangulations. In other words, the Delaunay triangulation is not the min-max angle triangulation.

In each case briefly explain your construction. Your example should be in general position (e.g., no four sites should be cocircular).

Hint: In both cases, it is possible build a counterexample consisting of just four points that are nearly co-circular. It suffices to present a single, specific example.

Exercise 4[2]

Suppose you want to compute a Voronoi data structure for a set of pixels drawn from an R-by-R grid. Devise an efficient scheme to support the following three operations.

- `create(R)`: create an empty data structure to handle an R-by-R grid.

[2]From Princeton COS 226 Final Exam.

- `insert(i,j)`: insert the pixel (i, j) into the data structure, where i and j are integers between 0 and $R - 1$.
- `find(x,y)`: return the inserted pixel which is closest to (x, y) in Euclidean distance, where x and y are integers between 0 and $R - 1$.

The `create` and `insert` operation should take $\mathcal{O}(R^2)$ time; the `find` operation should take $\mathcal{O}(1)$ time. For partial credit, give an algorithm that takes $\mathcal{O}(NR^2)$ time for `insert`, where N is the number of points already inserted into the data structure. (*Hint:* think simple.)

(a) Describe how to implement `create(R)`. Indicate what is being stored.

(b) Describe how to implement `find(i,j)`.

(c) Describe how to implement `insert(i,j)`.

Exercise 5

Assess the validity of the following claim: "If there exists an ϵ such that MaxClique is ϵ-approximable, then MaxClique is also $\frac{\epsilon}{4}$-approximable." If true, then give a $\frac{\epsilon}{4}$-appoximation for MaxClique. If false, provide a counterexample or reasoning to demonstrate why the claim does not hold.

Exercise 6

Prove that Chebyshev's inequality is tight: in other words, find a probability distribution for X and a value a such that

$$\Pr\left(|X - \mathrm{E}\left[X\right]| \geq a\right) = \frac{\mathrm{Var}\left(X\right)}{a^2}.$$

Hint: This random variable will take only three values.

Exercise 7

As in the homework, consider the general problem where we have m machines $M_1 \ldots M_m$ and a set of n jobs. Each job j has a processing time t_j. We seek to assign each job to one of the machines so that the loads placed on all machines

are as "balanced" as possible. That is, if $A(i)$ are the set of jobs assigned to machine M_i , then the load for machine M_i is total time required.

$$T_i = \sum_{j \in A(i)} t_j$$

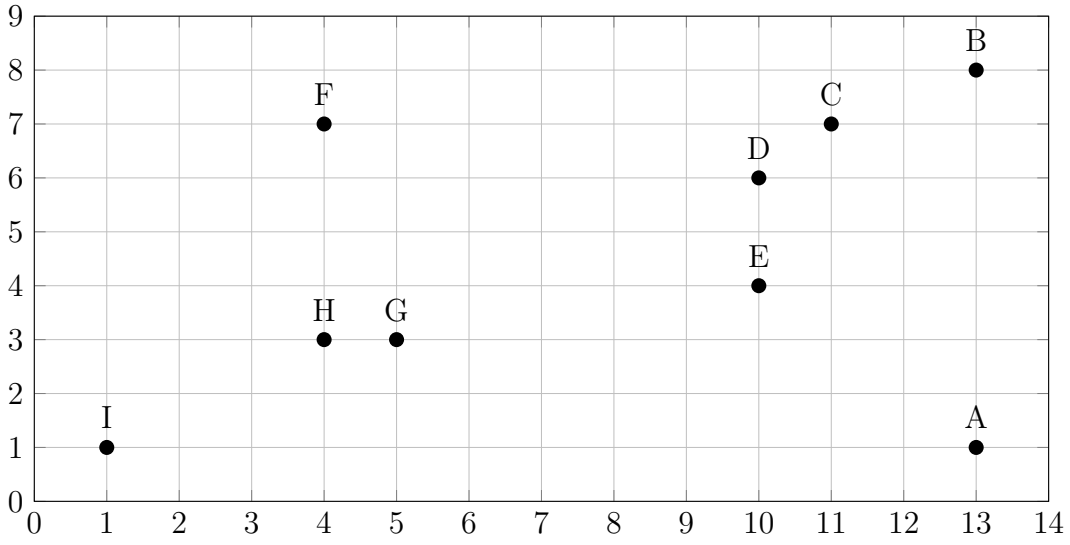
We want to minimize the *makespan* $T = \max_i T_i$.

(a) Describe a set of jobs such that the makespan of the greedy assignment is exactly $(2 - 1/m)$ times the makespan of the optimal assignment, where m is the number of machines.

(b) Describe an efficient algorithm to solve the minimum makespan scheduling problem exactly if every processing time T_i is a power of two.

Exercise 8

Run the Graham scan algorithm (as described in the lecture notes) to compute the convex hull of points below, starting from A. Give the points that appear on the trial hull (after each iteration) in the order that they appear.



Exercise 9

Again consider the following heuristic for constructing a vertex cover of a connected graph G: return the set of non-leaf nodes in any depth-first spanning

tree of G. We know (from the homework) this algorithm is a 2-approximation to the minimum vertex cover. Describe an infinite family of graphs for which this heuristic returns a vertex cover of size $2 \cdot OPT$.

Exercise 10

It is a fact that we still have no closed-form expression for the tail of the Normal distribution, which must be computed by numerically evaluating the integral. Let $X \sim \mathcal{N}(0, 1)$. Your goal is to produce upper bounds on $P(X \geq a)$, where $a > 0$.

(a) Use Markov's inequality to bound $\Pr(X \geq a)$. Note: This is not as trivial as it might seem because X is not non-negative. It will help to observe that:

$$\Pr(X \geq a) = \Pr(X \geq a \mid X > 0) \cdot \Pr(X > 0).$$

Now define the non-negative r.v. $Y = [X \mid X > 0]$ and note that $\Pr(X \geq a \mid X > 0) = \Pr(Y \geq a)$.

(b) Use Chebyshev's inequality to bound $\Pr(X \geq a)$.

(c) *(Optional)* Derive the Chernoff bound for $\Pr(X \geq a)$.

Exercise 11

Previously in this class, we discussed the **coupon collector** problem (do you remember when?) As a reminder: there are n distinct types of coupons you would like to collect. Each day you are sent a random coupon from among the n types. Let X denote the number of days needed to collect all n distinct coupons, given that coupons are chosen randomly with replacement. The following (famous) identity[3] is useful in answering some of the questions below:

$$\sum_{i=1}^{n} \frac{1}{i^2} = \frac{\pi^2}{6}.$$

(a) What is $\mathrm{E}[X]$?[4] What does this approach for high n? Write your answer using $\mathcal{O}(\cdot)$.

[3] known as the Basel problem.

[4] Of course we already discussed this in class, but good to practice!

(b) Derive $\mathrm{Var}(X)$. What does this approach for high n? Write your answer using $\mathcal{O}(\cdot)$.

(c) Derive an asymptotic upper bound on $\Pr(X \geq 2n\ln(n))$ for large n using Markov's inequality.

(d) Derive an asymptotic upper bound on $\Pr(X \geq 2n\ln(n))$ for large n using Chebyshev's inequality. Express your answer using $\mathcal{O}(\cdot)$.

Note: For $\mathrm{E}[X]$ in (c) and (d), use the asymptotic mean from part (a).

Exercise 12

(Erickson) Recall that a family $\mathcal{H}$ of hash functions is *universal* if, for all distinct items $x \neq y$,

$$\Pr_{h \in \mathcal{H}}[h(x) = h(y)] \leq \frac{1}{m}$$

where m is the size of the hash table. For any fixed hash function h, a collision is an unordered pair of distinct items x and y such that $h(x) = h(y)$.

Suppose we hash a set of n items into a table of size $m = 2n$, using a hash function h chosen uniformly at random from some universal family. Assume $\sqrt{n}$ is an integer.

(a) Prove that the expected number of collisions is at most $n/4$.

(b) Prove that the probability that there are at least $n/2$ collisions is at most $1/2$.

(c) Prove that the probability that any subset of more than $\sqrt{n}$ items all hash to the same address is at most $1/2$. [*Hint:* Use part (b).]

(d) *(Optional)* Now suppose we choose h at random from a *4-uniform* family of hash functions, which means for all distinct items w, x, y, z and all addresses i, j, k, l, we have

$$\Pr_{h \in \mathcal{H}}[h(w) = i \wedge h(x) = j \wedge h(y) = k \wedge h(z) = \ell] = \frac{1}{m^4}.$$

Prove that the probability that any subset of more than $\sqrt{n}$ items all hash to the same address is at most $\mathcal{O}(1/n)$.

Note: this part requires a more general form of Chebyshev's inequality to solve (which is why it is optional):

> **General Moment Inequality.** Let X be the sum of $2k$-wise independent r.vs. $X_1, X_2, \ldots, X_n$. Then for any fixed integer $k > 0$, $\Pr\left(|X - \mu|^k \geq z\right) = \mathcal{O}\left(\mu^k / z\right)$ for all $z > 0$. The hidden constant in the $\mathcal{O}(\cdot)$ notation depends on k. Rewriting this inequality we get $\Pr\left(X \geq (1+\delta)\mu\right) = \mathcal{O}\left(\left(1/\delta^2\mu\right)^k\right)$.

[*Hint:* All four statements have short elementary proofs via tail inequalities.]

Exercise 13

Let X be the number of fixed points of a random permutation on n elements. Calculate $\mathrm{E}[X]$ and $\mathrm{Var}(X)$ and use these to bound the probability that the number of fixed points of a random permutation is at least 10. Are Chernoff bounds useful to improve this?
e

Exercise 14

Consider the following problem P: Given a directed graph $G = (V, E)$ with weights $w_{i,j} \geq 0$ for each edge $(i, j) \in E$, the task is to partition V into two sets U and $W = V \setminus U$ so as to maximize the total weight of edges going from U to W, i.e., maximize

$$\sum_{\substack{(i,j)\in E \\ i\in U, j\in W}} w_{i,j}.$$

(a) Give a randomized $\frac{1}{4}$-approximation for the problem.

(b) Consider the following mixed integer linear program:

$$\begin{aligned} \text{maximize} \quad & \sum_{(i,j)\in E} w_{i,j} z_{i,j} \\ \text{s.t.} \quad & z_{i,j} \le x_i, && \text{for } (i,j) \in E, \\ & z_{i,j} \le 1 - x_j, && \text{for } (i,j) \in E, \\ & x_i \in \{0,1\}, && \text{for } i \in V, \\ & z_{i,j} \in [0,1], && \text{for } (i,j) \in E, \end{aligned}$$

Show that it solves P.

(c) *(Optional)* Consider a randomized rounding algorithm that solves the above mixed integer linear program and independently puts each vertex i into U with probability $\frac{1}{4} + x_i/2$. Prove that this gives a randomized $\frac{1}{2}$-approximation algorithm for P.

Exercise 15

(*Erickson*)

(a) The **chromatic number** $\chi(G)$ of an undirected graph G is the minimum number of colors required to color the vertices, so that every edge has endpoints with different colors. Computing the chromatic number exactly is NP-hard, because 3Color is NP-hard. Prove that the following problem is also NP-hard: Given an arbitrary undirected graph G, return any integer between $\chi(G)$ and $\chi(G) + 473$.

(b) Let $\alpha(G)$ denote the number of vertices in the largest independent set in a graph G. Prove that the following problem is NP-hard: Given a graph G, return any integer between $\alpha(G) - 31337$ and $\alpha(G) + 31337$.

Extra Practice

These are a collection of problems online I found interesting. I haven't drafted up solutions for these, and their relevance to the actual exam is undetermined.

Exercise 16

(Mount) This problem demonstrates an important application of computational geometry to the field of amphibian navigation. A frog who is afraid of the water wants to cross a stream that is defined by two horizontal lines $y = y^-$ and $y = y^+$. On the surface there are n circular lily pads whose centers are at the points $P = \{p_1, \ldots, p_n\}$. The frog crosses by hopping across the circular lily pads. The lily pads grow at the same rate, so that at time t they all have radius t (see Fig. 1(a)). Help the frog out by presenting an algorithm that in $O(n \log n)$ time determines the earliest time t^* such that there exists a path from one side of the stream to the other by traveling along the lily pads (see Fig. 1(b)).

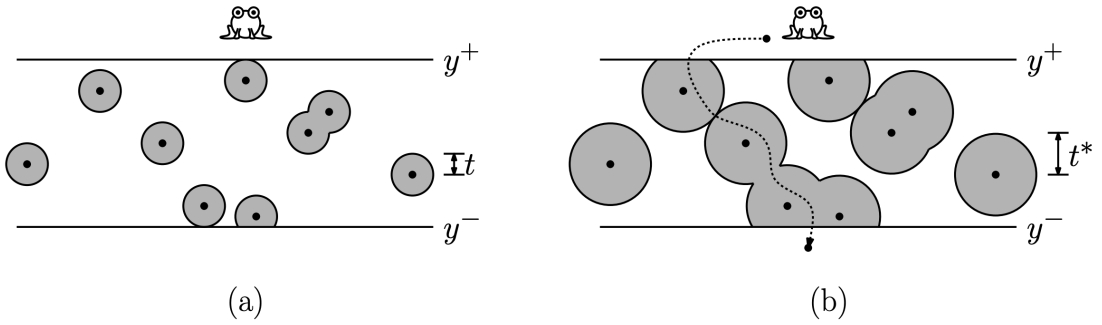


Figure 1

Exercise 17

You are given a collection of n non-intersecting circular disks in the plane, each of unit radius. Let $P = \{p_1, ..., p_n\}$ denote their center points. Given an arbitrary disk p_i in P as input, your task is to determine whether it is possible for this disk to escape from the others, meaning that it is possible to move this disk arbitrarily far away from the others without intersecting or moving any

of the other disks. Present an $O(n \log n)$ time algorithm to determine if it is possible for the disk to escape, and prove your algorithm is correct.

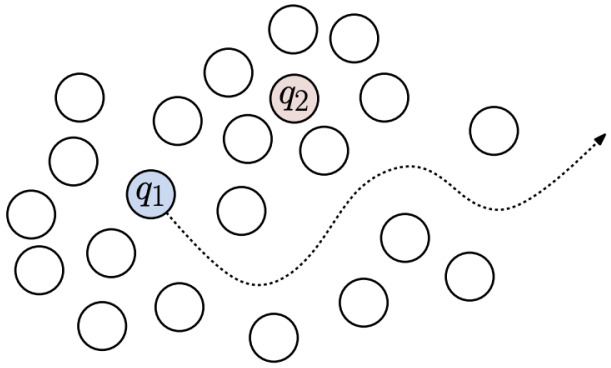


Figure 2: Here q_1 can escape, but q_2 cannot

Exercise 18

(Mount) The Delaunay triangulation of a convex polygon is defined to be the Delaunay triangulation whose sites are the vertices of the polygon. As usual, let us assume that the vertices $V = \{v_1, \ldots, v_n\}$ of the polygon are presented in counterclockwise order around the polygon. Also assume that $n \geq 3$ and no three vertices are collinear. In this problem, we will analyze a randomized incremental algorithm for constructing the Delaunay triangulation of a convex polygon.

The algorithm is similar to the randomized incremental algorithm given in class, but with the following differences. First, we do not use any sentinel sites, just the points themselves. We begin by permuting the points randomly. Let $P = (p_1, \ldots, p_n)$ denote the sequence of vertices after this permutation has been applied. We start with the triangle $\Delta p_1 p_2 p_3$. Then, we go through the points p_4 through p_n, adding each one and updating the Delaunay triangulation as we go. The insertion process for p_i involves the following steps:

i. Determine the edge ab of the current convex hull that is visible to p_i (see Fig. 1(a)).

ii. Connect p to the convex hull by adding the edges ap_i and $p_i b$ to the convex hull (see Fig. 1(b)).

iii. As in the notes, perform repeated edge flips until all the triangles incident to p_i satisfy the local Delaunay condition (see Fig. 1(c)).

Answer the following questions about this algorithm:

(a) Prove that in any triangulation of an n-sided convex polygon, the number of triangles is $n-2$ and the number of edges (including the edges of the convex hull) $2n-3$.

(b) What is the average degree of a vertex in the Delaunay triangulation of n? (Include the two edges of the convex hull that are incident to this vertex.) Derive your answer.

(c) Apply a backwards analysis to bound the expected number of edge flips performed when the ith site is inserted into the diagram (for $4 \leq i \leq n$).

(d) In step (i) of the above algorithm, we need to determine the edge ab of the convex hull that is visible to the new site p_i. Describe a method for answering these queries and derive its expected running time. (Hint: As we did in the standard Delaunay Triangulation algorithm, apply some form of bucketing combined with a backwards analysis.) $O(n \log n)$ total expected time is possible.

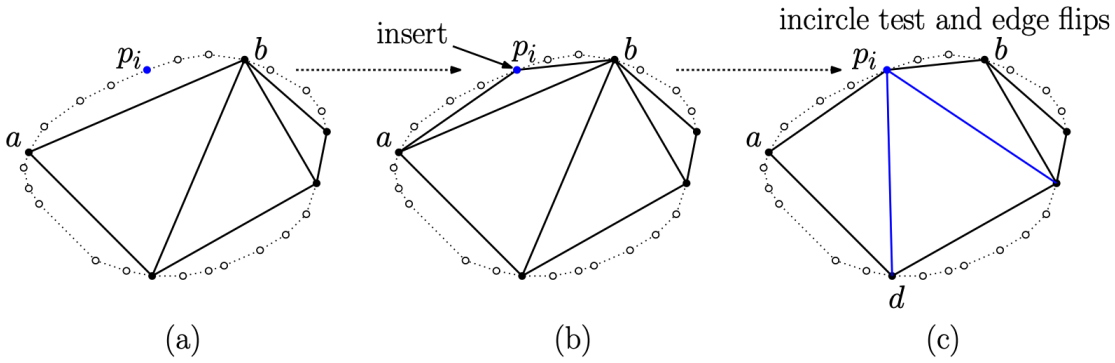


Figure 3: Delaunay triangulation of a convex point set.

Exercise 19[5]

You're the new owner of the restaurant chain Los Pollos Hermanos, and you want to expand your empire. You want to select a site for a new franchise which

[5]Yes, we covered this problem in class. But good to make sure you understand the solution.

is maximally far from all existing franchises to minimize competition and situate yourself near people who previously did not have a franchise nearby.

We can represent this problem as follows: given an n element point set P in $\mathbb{R}^2$ representing our franchises, we want to compute the largest circular disk that has its center in (or on) P's convex hull and contains no points of P in its interior (call such a disk *empty*). Describe and prove a $\mathcal{O}(n \log n)$ time algorithm that finds the empty disk of largest radius.

Hint: What can you say about the center of the disk?

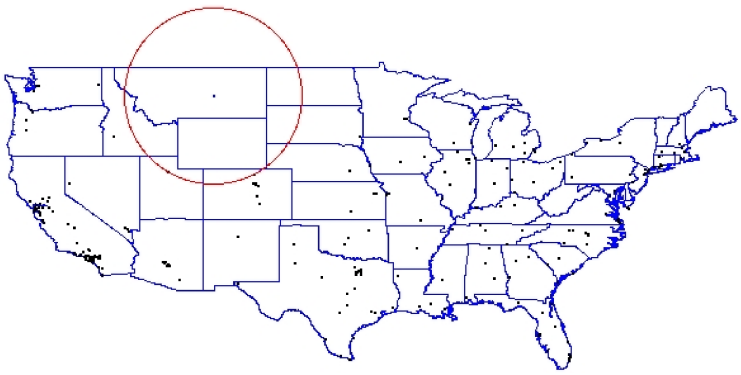


Figure 4: After your investigation, you find that your next Los Pollos Hermanos franchisee will be in Montana

Exercise 20

The following problem arises in telecommunications networks, and is known as the SONET ring loading problem. The network consists of a cycle on n nodes, numbered 0 through $n-1$ clockwise around the cycle. Some set C of calls is given; each call is a pair (i, j) originating at node i and ending at node j. The call can be routed either clockwise or counterclockwise around the ring. The objective is to route the calls so as to minimize the total load on the network. The load L_i on link $(i, i+1 \pmod n)$ is the number calls routed through the link, and the total load is $\max_{1 \le i \le n} L_i$. Give a 2-approximation algorithm for the SONET ring loading problem.